U24AI049

Sedha Patel

Assignment 5

AI202

Consider a problem that is stated as follows. Three girls and three boys are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side without ever leaving a group of girls in one place outnumbered by the boys in that place. Formulate the problem with appropriate search space and do as per following instructions.

1. Implement and solve the problem optimally using the following search problem.
   a) Depth Limited Search (take limit=3)
   b) Iterative Deepening Search

2. Give comparison of both algorithms with the number of explored states and state approximate time complexities.

```python
class Problem:
    def __init__(self):
        self.INITIAL = (3, 3, 1)

    def IS_GOAL(self, state):
        return state == (0, 0, 0)

    def ACTIONS(self, state):
        return [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

    def RESULT(self, state, action):
        g, b, boat = state
        dg, db = action
        if boat == 1:
            return (g - dg, b - db, 0)
        else:
            return (g + dg, b + db, 1)

    def IS_VALID(self, state):
        g, b, boat = state
        if not (0 <= g <= 3 and 0 <= b <= 3): return False
        if g > 0 and g < b: return False
        rg, rb = 3 - g, 3 - b
        if rg > 0 and rg < rb: return False
        return True

class Node:
    def __init__(self, state, parent=None, action=None, depth=0):
        self.STATE = state
        self.PARENT = parent
        self.ACTION = action
        self.DEPTH = depth
```

```python
def IS_CYCLE(node):
    curr = node.PARENT
    while curr:
        if curr.STATE == node.STATE: return True
        curr = curr.PARENT
    return False

def EXPAND(problem, node):
    for action in problem.ACTIONS(node.STATE):
        next_state = problem.RESULT(node.STATE, action)
        if problem.IS_VALID(next_state):
            yield Node(state=next_state, parent=node, action=action, depth=node.DEPTH + 1)
```

```python
def DEPTH_LIMITED_SEARCH(problem, l):
    frontier = [Node(problem.INITIAL)]
    cutoff_occurred = False
    explored_count = 0

    while frontier:
        node = frontier.pop()
        explored_count += 1

        if problem.IS_GOAL(node.STATE):
            return node, explored_count

        # If we reached the depth limit, do NOT expand further
        if node.DEPTH == l:         # L hai not 1(will get replaced by 3)
            cutoff_occurred = True
            continue

        if not IS_CYCLE(node):
            for child in EXPAND(problem, node):
                frontier.append(child)

    if cutoff_occurred:
        return "cutoff", explored_count
    else:
        return "failure", explored_count

# (Uses the same Problem, Node, IS_CYCLE, and EXPAND from Script 1)
```

```python
# (Uses the same Problem, Node, IS_CYCLE, and EXPAND from Script 1)

def ITERATIVE_DEEPENING_SEARCH(problem):
    total_states_explored = 0
    for depth in range(20):
        print(f"Searching depth limit: {depth}")
        result, count = DEPTH_LIMITED_SEARCH(problem, depth)
        total_states_explored += count

        if result != "cutoff" and result != "failure":
            return result, total_states_explored
    return "failure", total_states_explored

def print_path(node):
    path = []
    curr = node
    while curr:
        path.append(curr)
        curr = curr.PARENT
    print("\n--- Optimal Solution Path ---")
    for step in reversed(path):
        print(f"Depth {step.DEPTH}: {step.STATE} (Action: {step.ACTION})")

# Execution
river_problem = Problem()
print("--- RUNNING ITERATIVE DEEPENING SEARCH ---")
ids_result, total_count = ITERATIVE_DEEPENING_SEARCH(river_problem)

if isinstance(ids_result, Node):
    print_path(ids_result)
    print(f"\nTotal States Explored across all iterations: {total_count}")
else:
    print(f"Result: {ids_result}")
```

```
$ python -u "c:\Users\Shedha gami\AI202 lab\Assignment
--- RUNNING ITERATIVE DEEPENING SEARCH ---
Searching depth limit: 0
Searching depth limit: 1
Searching depth limit: 2
Searching depth limit: 3
Searching depth limit: 4
Searching depth limit: 5
Searching depth limit: 6
Searching depth limit: 7
Searching depth limit: 8
Searching depth limit: 9
Searching depth limit: 10
Searching depth limit: 11

--- Optimal Solution Path ---
Depth 0: (3, 3, 1) (Action: None)
Depth 1: (2, 2, 0) (Action: (1, 1))
Depth 2: (3, 2, 1) (Action: (1, 0))
Depth 3: (3, 0, 0) (Action: (0, 2))
Depth 4: (3, 1, 1) (Action: (0, 1))
Depth 5: (1, 1, 0) (Action: (2, 0))
Depth 6: (2, 2, 1) (Action: (1, 1))
Depth 7: (0, 2, 0) (Action: (2, 0))
Depth 8: (0, 3, 1) (Action: (0, 1))
Depth 9: (0, 1, 0) (Action: (0, 2))
Depth 10: (0, 2, 1) (Action: (0, 1))
Depth 11: (0, 0, 0) (Action: (0, 2))

Total States Explored across all iterations: 293
```

```
--- RUNNING DEPTH LIMITED SEARCH (LIMIT=3) ---
Result: cutoff
States Explored: 15

Shedha gami@LAPTOP-8UU5LAV5 MINGW64 ~
$
```